

AI-Powered Phishing Detector

Part 2 — High-Level Design, Tools & Development Phases

3. High-Level Design of the Proposed Solution

3.1 Overview of the System Architecture

The AI-powered phishing detector is organised as a modular, layered pipeline. Each layer has a clearly defined responsibility and communicates structured data to the next stage. The system operates in two distinct modes: an offline training mode that builds and saves model artefacts, and an online prediction mode that loads those artefacts to serve real-time classification requests through a Flask web API.

The pipeline implements two parallel feature and model versions: a Baseline pipeline (V1 — TF-IDF + 7 handcrafted phishing signals, trained with Complement Naive Bayes and Logistic Regression) and an improved V2 pipeline (TF-IDF + 9 refined signals including legitimacy indicators, with class balancing). At inference time, a hybrid Ensemble Override layer combines the V2 model score with rule-based legitimacy correction to address the false-positive problem observed on legitimate security notifications and institutional emails.

3.2 Main System Components

The system is structured into five primary layers, each mapping to concrete implemented modules:

#	Layer	Actual Module(s)	Input	Output
1	Data Layer	phishing_legitimate.csv spam_assassin.csv	Raw CSV files	Labelled email records
2	Preparation Layer	data_preparation.py	Raw CSVs (heterogeneous schemas)	Cleaned_emails.csv — 5,796 rows, unified binary labels

3	Feature Layer	feature_engineering.py (V1) feature_engineering_v2.py (V2)	Cleaned text + metadata	Feature matrix: TF-IDF + 7 (V1) or 9 (V2) custom signals
4	Model Layer	model_training.py / model_training_v2.py predictor.py predictor_v2_improved.py predictor_ensemble.py	Feature matrix + labels	Trained artefacts + phishing probability score
5	Serving Layer	app.py (Flask REST API) index.html (Web UI)	Raw email text (pasted via UI or POST /predict)	JSON: label, confidence, phishing_probability, reasoning signals

3.3 Component Descriptions

3.3.1 Data Layer

Two public datasets are used as input sources:

- phishing_legitimate.csv — balanced phishing vs. legitimate email samples. Uses label column for classification target.
- spam_assassin.csv — SpamAssassin corpus providing additional email variety. Uses target column for classification target.

Note: Input during inference is raw email text pasted directly into the web UI or submitted via the POST /predict API endpoint. No live email server (Gmail API, Outlook API) integration is used.

3.3.2 Preparation Layer — data_preparation.py

This module handles schema heterogeneity between both datasets automatically:

- Auto-detects text and label columns across both CSVs regardless of column naming (label vs. target).
- Harmonises labels to unified binary format: 1 = phishing / spam, 0 = legitimate / ham.
- Drops invalid or empty rows and merges both sources.
- Outputs cleaned_emails.csv: 15,796 rows (~7,900 legitimate, ~7,896 phishing — balanced dataset).

3.3.3 Feature Layer — V1 Baseline vs. V2 Improved

Two feature pipelines are implemented, trained, and compared:

Feature	V1 — feature_engineering.py	V2 — feature_engineering_v2.py
Text Representation	TF-IDF unigrams, top vocabulary	TF-IDF unigrams + bigrams, extended vocabulary
Phishing Signals	suspicious_word_count: 21 generic terms (urgent, verify, login, click, account, suspended, password...)	phishing_keyword_count: 24 multi-word phrases (verify account, click here, limited time, update payment, claim reward...)
Legitimacy Signals	Not present	legitimate_keyword_count: 12 professional phrases (best regards, please see attached, meeting notes, team member, director...)
Greeting Detection	Not present	has_generic_greeting: binary flag for dear sir / dear madam / dear user
Signature Detection	Not present	has_signature: binary flag for regards sincerely / best / thank you
Shared Signals	url_count, email_address_count, has_urgency, caps_ratio, text_length, exclamation_count	Same 6 signals — identical extraction method ensures V1/V2 comparability
Class Balancing	None	class_weight='balanced' applied in Logistic Regression training
Total Custom Features	7 features	9 features

No training/serving skew: The clean_email() preprocessing function is implemented identically across data_preparation.py, predictor.py, predictor_v2_improved.py, and predictor_ensemble.py — confirmed by code audit.

3.3.4 Model Layer — Baseline, V2, and Ensemble

Three inference modules are implemented at increasing sophistication levels:

- predictor.py — Baseline: Complement Naive Bayes or Logistic Regression on V1 features. Fast and interpretable. Achieves ~99% accuracy on test set.
- predictor_v2_improved.py — V2: Uses V2 feature pipeline with class balancing. Improves precision on ambiguous cases such as password-reset notifications and institutional emails.
- predictor_ensemble.py — Production: Hybrid model combining V2 ML probability with rule-based legitimacy override. This is the module served by the Flask API. Decision threshold set to 0.70 (tuned via analyze_threshold.py to optimise precision-recall balance).

Ensemble Logic: The ensemble predictor first computes the V2 ML phishing probability. It then checks for legitimacy evidence (professional signature, organisational context, non-phishing password-notification patterns). When explicit phishing patterns are detected, the legitimacy override is suppressed (multiplied by 0.3) to prevent false negatives. Final label is determined by comparing the adjusted probability against the 0.70 threshold.

3.3.5 Serving Layer — Flask API + Web UI

The Flask application (app.py) exposes two endpoints:

- /predict (POST) — accepts raw email text, runs the ensemble predictor via lazy-loaded artefacts (vectorizer_v2.pkl, model_v2.pkl), returns JSON: { label, confidence, phishing_probability, reasoning_signals }. Returns HTTP 503 if artefacts are missing, HTTP 500 on runtime errors.
- / (GET) — serves the HTML/JS single-page web interface. Users paste email text and receive a verdict with confidence score and signal breakdown. Defensive null checks prevent rendering errors on edge-case responses.

Confirmed deliverable: The Flask web application is a fully implemented and tested executable component of this project, not an optional extension.

3.4 System Functional Flow

3.4.1 Offline Training Flow

Step	Script	Action
1	data_preparation.py	Load both CSVs, auto-detect schemas, unify labels (spam/phishing → 1, ham/legitimate → 0), output cleaned_emails.csv (15,796 rows, balanced).
2	feature_engineering.py	Build V1 feature matrix: TF-IDF + 7 handcrafted signals. Save X_features.npz and y_labels.npy.
3	feature_engineering_v2.py	Build V2 feature matrix: TF-IDF + 9 signals including legitimacy indicators. Save X_features_v2.npz and y_labels_v2.npy.
4	model_training.py	Train Complement Naive Bayes and Logistic Regression on V1 features. Evaluate and save best model as model.pkl + vectorizer.pkl.
5	model_training_v2.py	Train with class_weight='balanced' on V2 features. Save best model as model_v2.pkl + vectorizer_v2.pkl. These are the production artefacts.

6	<code>analyze_threshold.py</code>	Utility: compute precision/recall curves across decision thresholds. Confirms 0.70 as optimal production threshold for the V2 model.
---	-----------------------------------	--

3.4.2 Online Prediction Flow (Ensemble)

Step	From	To	Data / Action
1	User (UI or API)	Flask /predict	POST: raw email text string
2	Flask app.py	<code>predictor_ensemble.py</code>	Lazy-load <code>vectorizer_v2.pkl</code> + <code>model_v2.pkl</code> on first request. Pass raw text to ensemble predictor.
3	<code>predictor_ensemble.py</code>	Preprocessing (<code>clean_email</code>)	Strip HTML, mask URLs/emails, lowercase, remove noise → cleaned text string.
4	Preprocessing	V2 Feature Extractor	Cleaned text → TF-IDF vector + 9 custom feature signals → combined feature vector.
5	V2 Feature Extractor	V2 ML Classifier	Feature vector → Logistic Regression (<code>model_v2.pkl</code>) → raw phishing probability (0.0–1.0).
6	V2 ML Classifier	Ensemble Override	Raw probability + feature flags → <code>check_legitimacy_signals()</code> → adjust probability. Override suppressed (×0.3) if phishing patterns detected.
7	Ensemble Override	Decision Module	Adjusted probability compared to threshold 0.70 → final label: Phish (≥0.70) or Ham (<0.70).
8	Decision Module	User + Audit Log	JSON response: { label, confidence, phishing_probability, reasoning_signals }.

3.5 Design Trade-offs and Rationale

Decision	Choice Made	Rationale
Primary classifier	Complement Naive Bayes + Logistic Regression	CNB outperforms standard NB on imbalanced text corpora. LR provides interpretable coefficients and probability calibration. Both deliver <10ms inference — suitable for real-time API serving.

Why not BERT	Not used as primary model	BERT inference takes ~200ms per email and requires GPU for production. The V2 LR pipeline achieves ~99% accuracy on the test set at a fraction of the compute cost. BERT is listed as a future enhancement.
Dual feature pipelines	V1 (7 features) + V2 (9 features) in parallel	Enables direct comparison between generic and domain-aware features. V2 legitimacy signals specifically target the false-positive problem on institutional and password-reset emails identified during evaluation.
Ensemble override	Rule-based legitimacy correction at inference time	Offline test accuracy (~99%) does not capture production false-positive rates. The override layer corrects for dataset bias (underrepresentation of legitimate security-notification emails) without requiring retraining.
Decision threshold: 0.70	Raised from default 0.50 to 0.70	analyze_threshold.py confirms that threshold 0.50 produces unacceptable false-positive rates on password-reset emails. 0.70 optimises the precision-recall trade-off for this deployment context.
Class balancing	class_weight='balanced' in V2 training	Prevents the classifier from over-fitting to the majority class. Applied only in V2 to allow fair comparison with the unbalanced V1 baseline.

4. Tools and Technologies

All tools listed below are present in requirements.txt and actively used in the implemented pipeline:

Category	Tool	Version	Specific Role in This Project
Language	Python	3.11	Core language for all pipeline stages: data preparation, feature extraction, model training, and API serving.
Data	Pandas	2.x	Schema-agnostic CSV loading in data_preparation.py; auto-detect label/target columns; output cleaned_emails.csv.
	NumPy	1.26	Sparse feature matrix operations (X_features_v2.npz); label array storage (y_labels_v2.npy).

	SciPy	1.4.x	Sparse matrix format (scipy.sparse) used internally by Scikit-learn TF-IDF and saved feature artefacts.
NLP & Features	NLTK	3.8	Text normalisation in clean_email(): HTML tag stripping, tokenisation, stop-word removal, lemmatisation.
	Scikit-learn	0.24	TfidfVectorizer, ComplementNB, LogisticRegression, train_test_split, classification_report, confusion_matrix.
	Regex (re)	stdlib	URL masking, email address extraction, phishing multi-word pattern matching, sender domain analysis.
Artefacts	joblib / Pickle	stdlib	Serialise vectorizer_v2.pkl and model_v2.pkl for lazy-loading in Flask serving layer.
Visualisation	Matplotlib	3.8	Confusion matrices, precision-recall threshold curves (analyze_threshold.py), class distribution plots.
	Seaborn	0.13	Feature correlation heatmaps, training metric comparison charts in EDA notebook.
Serving	Flask	2.0.x	REST API: GET / (HTML UI), POST /predict (JSON verdict). Lazy artefact loading, HTTP 503/500 error handling.
	HTML / JavaScript	—	Single-page web UI served by Flask: email textarea, async fetch to /predict, null-safe verdict rendering.
Dev Environment	Jupyter Notebook	7.x	EDA (01_eda.ipynb): class distribution, top phishing vocabulary, feature correlation, data balance analysis.
	Visual Studio Code	—	Source code editing, debugging, integrated Git operations.
Version Control	Git + GitHub	—	Repository hosting all source modules, datasets, model artefacts, notebooks, and documentation.

5. Development Phases

The project is structured into nine sequential phases. Each phase has defined inputs, outputs, and responsible team members.

#	Phase	Duration	Deliverables	Responsible
1	Problem Definition & Setup	Week 1	GitHub repository, requirements.txt (11 packages), confirmed evaluation metrics (Precision, Recall, F1, AUC), initial README	All members
2	Data Collection & Preparation	Week 2	phishing_legitimate.csv + spam_assassin.csv acquired; data_preparation.py producing cleaned_emails.csv (15,796 rows, balanced); class distribution analysis	ML Engineer
3	V1 Feature Engineering & Baseline Model	Week 3–4	feature_engineering.py (7 signals); model_training.py (CNB + LR); artefacts model.pkl + vectorizer.pkl; evaluation report (accuracy, confusion matrix, precision/recall/F1)	NLP Specialist + ML Engineer
4	V2 Feature Engineering & Improved Model	Week 5	feature_engineering_v2.py (9 signals + legitimacy indicators); model_training_v2.py with class_weight='balanced'; artefacts model_v2.pkl + vectorizer_v2.pkl; side-by-side comparison vs V1 baseline	NLP Specialist
5	Threshold Tuning & Ensemble Logic	Week 6	analyze_threshold.py confirming threshold 0.70 as optimal; predictor_ensemble.py with legitimacy override suppression logic; false-positive analysis on password-reset and institutional emails	ML Engineer + Backend Dev
6	Flask API & Web Interface	Week 7	app.py with GET / and POST /predict endpoints; lazy artefact loading; HTTP error handling (503/500); HTML/JS single-page UI with null-safe verdict rendering	Backend + Frontend Dev
7	EDA Notebook	Week 7	notebooks/01_eda.ipynb: class distribution chart, top phishing vocabulary, feature correlation matrix, text length distribution, dataset balance confirmation	NLP Specialist
8	Testing & Validation	Week 8	tests/test_api.py: password-reset email (main false-positive case), classic phishing email, organisational email; tests/test_v2_threshold.py: threshold sensitivity at 0.50–0.90; false positive/negative analysis report	QA / Project Lead

9	Documentation & Demo	Week 9	Final report, updated README (run instructions for all scripts), live demo via web UI covering 3 test scenarios (phishing, password reset, organisational email), GitHub repository final cleanup	All members
---	----------------------	--------	---	-------------

5.1 Team Roles

#	Role	Responsibilities
1	ML Engineer	data_preparation.py, V1 and V2 model training, class balancing, analyze_threshold.py, artefact management
2	MLP Specialist	feature_engineering.py and feature_engineering_v2.py, TF-IDF tuning, phishing/legitimacy signal design, EDA notebook (01_eda.ipynb)
3	Backend Developer	app.py Flask API, predictor_ensemble.py integration, lazy-loading, error handling, /predict endpoint, JSON response structure
4	Frontend Developer	HTML/JS web UI, verdict display, confidence rendering, null-safe API response handling
5	Project Lead / QA	GitHub management, tests/test_api.py and tests/test_v2_threshold.py, false-positive analysis, final documentation, presentation

5.2 GitHub Repository Structure

Repository: <https://github.com/AminMnari/AI-Powered-Phishing-Email-Detector>

Path	Contents
data/	phishing_legitimate.csv, spam_assassin.csv, cleaned_emails.csv, X_features_v2.npz, y_labels_v2.npy
models/	model_v2.pkl (Logistic Regression, balanced), vectorizer_v2.pkl (TF-IDF V2), model.pkl (V1 baseline), vectorizer.pkl (V1)
notebooks/	01_eda.ipynb — EDA: class distribution, vocabulary analysis, feature correlations, dataset balance
src/data_preparation.py	Schema normalisation, label harmonisation, cleaned_emails.csv generation
src/feature_engineering.py	V1 feature extractor: TF-IDF + 7 handcrafted phishing signals
src/feature_engineering_v2.py	V2 feature extractor: TF-IDF + 9 signals including legitimacy indicators

src/model_training.py	V1 model training: CNB + LR on V1 features, saves model.pkl + vectorizer.pkl
src/model_training_v2.py	V2 model training: LR with class balancing on V2 features, saves model_v2.pkl + vectorizer_v2.pkl
src/predictor.py	V1 baseline inference module
src/predictor_v2_improved.py	V2 improved inference module
src/predictor_ensemble.py	Production hybrid ensemble: V2 ML score + legitimacy override logic, threshold 0.70
src/analyze_threshold.py	Utility: threshold sensitivity analysis, precision-recall curves
src/app.py	Flask REST API + web UI server (GET /, POST /predict)
tests/test_api.py	3 test scenarios: classic phishing, password-reset (false-positive risk), organisational email
tests/test_v2_threshold.py	Threshold sensitivity test: same email at 0.50, 0.60, 0.70, 0.80, 0.90
requirements.txt	11 Python dependencies for full pipeline reproduction
README.md	Setup, run instructions for all scripts, demo guide, team members